Facade

Facade · One placeOrder call over inventory+payment+shipping · Build with Waqa

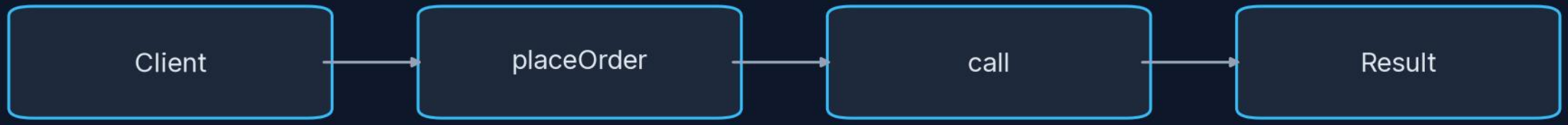
Today's Hook

- Problem: One placeOrder call over inventory+payment+shipping
- Pattern: Facade — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- One placeOrder call over inventory+payment+shipping
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Facade: One placeOrder call over inventory+payment+shippin

Real-World Scenario

- Running example: One placeOrder call over inventory+payment+shipping
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.gof.structural.facade;
import com.buildwithwaqar.patterns.gof.structural.facade.OrderFacade;
/** Problem: place an order without teaching every caller inventory+payment+shipping APIs. */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== Facade problem: one-call checkout ===");
    var f = new OrderFacade();
    System.out.println(f.placeOrder("sku-1", 1, "4111", 1500));
    System.out.println(f.placeOrder("", 1, "4111", 1500));
  }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.gof.structural.facade;

final class Inventory { boolean reserve(String sku, int qty) { return qty > 0 && !sku.isBlank(); } }
final class PaymentGateway { boolean charge(String card, int cents) { return card != null && cents > 0; } }
final class Shipping { String ship(String sku) { return "TRACK-" + sku; } }

/** Simple interface over a noisy subsystem. */
public final class OrderFacade {
    private final Inventory inventory = new Inventory();
    private final PaymentGateway payments = new PaymentGateway();
    private final Shipping shipping = new Shipping();

    public String placeOrder(String sku, int qty, String card, int cents) {
        if (!inventory.reserve(sku, qty)) return "REJECTED:inventory";
        if (!payments.charge(card, cents)) return "REJECTED:payment";
        return "OK:" + shipping.ship(sku);
    }
}
```

Runnable main

```java
package com.buildwithwaqar.patterns.gof.structural.facade;
import com.buildwithwaqar.patterns.gof.structural.facade.OrderFacade;
/** Problem: place an order without teaching every caller inventory+payment+shipping APIs. */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== Facade problem: one-call checkout ===");
    var f = new OrderFacade();
    System.out.println(f.placeOrder("sku-1", 1, "4111", 1500));
    System.out.println(f.placeOrder("", 1, "4111", 1500));
  }
}
```

Walkthrough Output

```
=== Facade problem: one-call checkout ===
OK:TRACK-sku-1
REJECTED:inventory
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Facade.

Related Patterns

- Contrast Facade with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Facade → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.